

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 735 — Parallel Computing

Multicore, SIMD, GPU, and distributed memory — with a performance mindset and honest measurement

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 4 (with CSCE 650 and CSCE 748)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 614 Computer Architecture; fluency in C or C++

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A real workload parallelised three ways — threads, GPU, and distributed — each with measured scaling, a roofline placement, and an honest account of where the speedup stopped

Course description

Every other course in this program produced something that worked. This one is about making it fast, which turns out to be a different discipline with a different failure mode: **code that is correct, parallel, and no faster.**

The organising fact is that **single-thread performance stopped improving** around 2005. Dennard scaling ended, clock speeds plateaued, and the transistors kept coming — so they went into more cores, wider vectors, and specialised units. Performance is still available and it is no longer free. Getting it requires restructuring the computation, and that is what this course teaches.

The second fact is that **almost nothing is compute-bound**. The overwhelming majority of real workloads are limited by memory bandwidth, by synchronisation, or by load imbalance — and optimising the arithmetic in a bandwidth-bound loop achieves nothing. **Module 07's roofline model exists to tell you which problem you have** before you spend a week on the wrong one.

Throughout, the discipline is measurement. Parallel programming is unusually rich in confident claims that do not survive a benchmark — superlinear speedups from cache effects, speedups measured against an unoptimised baseline, scaling curves that stop exactly where they would have bent. **This course measures everything and reports what it finds.**

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Apply Amdahl's and Gustafson's laws and know which applies.

- 2 Distinguish strong from weak scaling and measure both.
- 3 Write correct shared-memory parallel code and reason about the memory model.
- 4 Use atomics and lock-free techniques, and say when they are not worth it.
- 5 Decompose a problem and balance the load dynamically.
- 6 Build a roofline model and use it to decide what to optimise.
- 7 Write and optimise CUDA kernels.
- 8 Write distributed-memory programs with MPI and overlap communication.
- 9 Implement the standard parallel primitives: scan, reduce, sort.
- 10 Test concurrent code, and know what testing cannot establish.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Why Parallel, and What It Costs	Why is everything parallel now, and what does it buy?
2	Parallel Architectures	What hardware are you actually targeting?
3	Threads and Shared Memory	How do you write parallel code that is correct?
4	Synchronisation and Memory Models	Why does your correctly locked program still misbehave?
5	Lock-Free Programming	When is lock-free worth the difficulty?
6	Decomposition and Load Balancing	How do you split a problem so all the cores stay busy?
7	Performance Analysis	What is limiting this code, and how would you know?
8	GPU Programming	How do you program a throughput processor?
9	GPU Optimisation	How do you make a correct kernel a fast one?
10	Distributed Memory and MPI	How do you compute across machines that share nothing?
11	Parallel Algorithms	What does a parallel algorithm look like?
12	Patterns, Frameworks, and Portability	Can you write parallel code that outlives one machine?
13	Correctness, Debugging, and Shipping	How do you know parallel code is right?

Projects

Project 1 — Parallelise something real, on the CPU (after Module 7)

Take a genuine workload — ideally one of your own from another course in this program — and make it fast on a multicore machine. Measurement is the deliverable, not the speedup.

Requirements

- A profiled, optimised *serial* baseline. **Parallelising unoptimised code and reporting the speedup is the classic dishonesty** and is not accepted here.
- A threaded implementation with a stated decomposition and a stated synchronisation strategy.
- Strong scaling measured from 1 to the maximum available threads, plotted.
- Weak scaling measured over the same range, plotted separately.
- A roofline placement of the kernel, with the measured arithmetic intensity.
- A thread sanitiser run showing no data races.

Done means

- **Two scaling plots, strong and weak, with the point where each departs from ideal identified and explained.**
- A roofline plot with the kernel placed on it, and a statement of whether it is compute- or bandwidth-bound.
- **Speedup against the *optimised* serial baseline**, with that baseline's optimisation documented.
- **An honest account of where the parallelisation stopped helping and why.** Every workload has that point; finding it is the exercise.

Project 2 — GPU or distributed (after Module 12)

Take the same workload further. One platform, done thoroughly, with the optimisation steps measured individually.

Requirements

- **Either** a CUDA implementation with at least three optimisation steps applied and measured separately — coalescing, shared memory tiling, occupancy tuning; **or** an MPI implementation with communication/computation overlap and a scaling study across nodes.
- A profiler report identifying the limiting resource at each stage.
- A roofline placement for the target platform.
- Correctness verified against the serial result, bitwise where the arithmetic permits and within tolerance where it does not.
- A comparison against a well-tuned library implementation if one exists.

Done means

- **A table of each optimisation step with its measured effect**, including any that made things worse.
- A profiler-supported statement of what limits the final version.
- **An honest comparison against cuBLAS, Thrust, or an equivalent library. You will probably lose.** Report by how much and explain where the gap is.
- A statement of the largest problem size the implementation handles and what fails beyond it.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
CMU 15-418 / Stanford CS149 — Parallel Computer Architecture and Programming (free)	The primary source. Complete lectures, slides, and assignments. Kayvon Fatahalian's lectures are the best free treatment of the architecture-to-programming connection that exists.
CMU 15-418 course site (free)	The same course at CMU, with a different assignment set. Worth having both for the exercises.
NVIDIA CUDA C++ Programming Guide (free)	The reference for Modules 08 and 09. Dense, authoritative, and the Best Practices Guide alongside it is the practical companion.
OLCF CUDA Training Series (free video and exercises)	A structured, hands-on CUDA course from Oak Ridge. Good pacing, and the exercises run on hardware you have.
Herlihy & Shavit — The Art of Multiprocessor Programming	The reference for Modules 04 and 05. Library copy; the authors' lecture slides are free and cover the key material.
Victor Eijkhout — Introduction to High Performance Scientific Computing (free book)	Free, comprehensive, and strong on MPI and on the numerical side. The reference for Module 10.

Tooling

- **C++17 with OpenMP and `std::thread`.** The memory model work in Module 04 needs a language that has one.
- **A machine with at least 4 cores**, and preferably more. Scaling curves from a 2-core machine are not informative.
- **An NVIDIA GPU** for Modules 08 and 09. **A laptop GPU is sufficient** — the techniques are the same and the numbers are smaller. Cloud instances are an alternative.
- **A profiler you trust:** perf or VTune on CPU, Nsight Compute on GPU. **Performance claims without a profile are guesses**, and this course does not accept them.
- **MPI** — OpenMPI or MPICH. Multiple processes on one machine is adequate for correctness; borrowed nodes or a cloud cluster for the scaling study.
- **A plotting setup.** You will produce scaling curves and roofline plots constantly, and they are the deliverable.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.